UNITED INTERNATIONAL UNIVERSITY (UIU)
Dept. of Computer Science & Engineering
Trimester: Summer 2026 (Online-2)
Course No: CSE 3812 **Title:** Artificial Intelligence Laboratory
Section: H **Deadline:** 7th September, 2026, 11:59PM **Marks:** 10

Instructions:

- You will need to use the AC-3 (Arc Consistency-3) algorithm for preprocessing. The pseudocode is provided in Annex. You can look it up online for further understanding.
- Even though I said otherwise, the use of LLMs without understanding what it's returning is a common scene from the last assignment. So, the viva will be reinforced from now on. Make sure you are prepared to explain every single thing in your code.
- The best way to get full marks in the assignment(s) is to code yourself. Trust me, you'll learn a lot.

Deliverables Instructions:

- The source code, input, output and technical report all should be put in a single folder and then zip the folder.
- The folder name will be your `rollNumber_CSPAssignment` e.g. for me, it will be `1907016_CSPAssignment`
- Upload the zip folder in this current eLMS submission post. (Your upload is limited to one submission only)
- In the next lab, bring the printed copy of the report along with yourself for the conduction of your viva.
- Late submissions are allowed but you will be graded out of 80% for the total marks.

Intelligent Sudoku Solver

You are tasked with developing an Artificial Intelligence solver in Python that models the classic Sudoku puzzle strictly as a CSP. You must abandon naive brute-force architectures and instead implement logical state-space representations, Backtracking Search, domain-reducing constraint propagation algorithms, and the Minimum Remaining Values (MRV) heuristic.

Problem Definition and Constraints:

Sudoku is defined as a 9x9 grid where each independent cell must be assigned an integer from 1 to 9. The system reaches a valid solution when all 81 variables are assigned such that no integer is duplicated within any horizontal row, vertical column, or any of the nine pre-defined 3x3 subgrids. You will receive puzzle data representing varying degrees of difficulty, demanding an algorithm that dynamically prunes the search space to avoid combinatorial explosion.

Input Data Specification:

To ensure standardized automated grading, your application must read input from a plain text file (e.g., puzzles.txt). Each line within this file will represent a single, independent Sudoku puzzle.

- A puzzle is formatted as a continuous, 81-character alphanumeric string.
- The string maps to the board horizontally; characters 1-9 represent the top row, characters 10-18 represent the second row, and so forth.
- Pre-assigned givens are represented by their respective digits 1 through 9.
- Unassigned variables are explicitly represented by the digit 0.

Output Data Specification:

Your solver must output its results to a newly generated file named solutions.txt. For every puzzle processed from the input file, your code must write an 81-character string to the output file representing the fully solved, consistent board state. No 0 digits should exist in the output string.

Deliverables:

You must submit your fully commented Python source code (e.g., search.py / search.ipynb) along with a concise technical report. The report must contain an ablation study of your algorithm. Specifically, you must execute your solver on the provided "Hard" difficulty puzzle both with the MRV heuristic enabled and with the MRV heuristic disabled (defaulting to sequential variable selection). You must mathematically compare the execution time and the total number of node expansions (backtracks) required under both conditions.

Grading:

The grading rubric for this assignment focuses heavily on algorithmic correctness and OOP architecture over raw execution time, provided the solver correctly utilizes the taught heuristics.

- Twenty percent of the final grade is allocated to the accuracy of the CSP instantiation parsing.
- Fifteen percent is dedicated to constraint validation logic (checkAxes, checkIfUsed).
- Twenty percent is allocated to the correct execution of the AC-3 Revise queue logic.
- Twenty percent is assigned to the functional stability of the recursive Backtracking framework.
- Fifteen percent relies upon the successful deployment of the MRV heuristic.
- The final ten percent evaluates adherence to input/output formatting and the depth of the ablation study within the technical report.

Annex:

AC-3 Pseudocode:

```
function AC-3(csp) returns false if an inconsistency is found, true otherwise
    inputs: csp, a binary CSP with components (X, D, C)
    local variables: queue, a queue of arcs, initially all the arcs in csp

    while queue is not empty do
        (Xi, Xj) ← REMOVE-FIRST(queue)
        if REVISE(csp, Xi, Xj) then
            if size of Di = 0 then return false
            for each Xk in Xi.NEIGHBORS - {Xj} do
                add (Xk, Xi) to queue
    return true


function REVISE(csp, Xi, Xj) returns true iff we revise the domain of Xi
    revised ← false
    for each x in Di do
        if no value y in Dj allows (x, y) to satisfy the constraint between Xi and Xj then
            delete x from Di
            revised ← true
    return revised
```

HAPPY CODING

<_____| 🌷 |————————>